

ENTERPRISE NTFS ACL LAB

NovaSec Corp File Server Simulation

[Security Descriptors](#) · [DACL](#) · [SACL](#) · [SIDs](#) · [Service Accounts](#) · [Active Directory](#)

Field	Value
Author	Ihar Shcharbitski
Date	28.05.2026
Environment	Oracle VirtualBox 7.2.8 — Windows 11 Enterprise Evaluation
Classification	Personal Learning Documentation

1. Objective & Scenario

This lab simulates the file server of NovaSec Corp — a small company with three human roles and three automated services. I will design, enforce, audit, and intentionally break an access control system from scratch, using the same process a security engineer follows when hardening a production file server.

The goal is to understand not just what to configure, but why Windows works the way it does at the security architecture level — covering the full stack from Security Descriptors down to individual ACE propagation flags.

1.1 Concepts Covered

Concept	Why It Matters in the Real World
Security Descriptors	Every Windows object (file, process, service, registry key) carries one — it is the container for all access control.
Security Identifiers (SIDs)	Windows never trusts usernames internally — it operates on SIDs. Understanding SID structure is essential for reading ACLs and audit logs.
DACL & ACE Structure	The actual rules governing access. Each <code>icacls</code> command creates or modifies one ACE inside the DACL.
Inheritance Chains	How permissions propagate through a folder tree — and how to intentionally break that chain to create hard security isolation.
Service Account Permissions	Automated processes need tightly scoped access. Misconfigured service accounts are one of the most common real-world attack surfaces.

1.2 NovaSec Corp — Principal Roster

Principal	Type	Access Requirement
Ihar_Admin	Human / Admin	Full control — security engineer and lab owner
SecAdmins	Group	Admin group — elevated rights across the entire tree
Analyst	Human	Reads and modifies project files within defined scope
Analysts	Group	Read all Projects, modify Internal only
Guest_Lab	Human	External contractor — read Public folder only
Guests	Group	Read Public, no access elsewhere
WebAppSvc	Service Account	Internal web application — read-only on Projects\Public
BackupSvc	Service Account	Nightly backup job — read-only across the entire tree
AuditLogger	Service Account	SIEM log forwarder — write-only to Logs, no read access elsewhere

2. Environment Setup

2.1 System Information

Field	Value
OS Version	Windows 11 Enterprise Evaluation (winver)
OS Build	26200.6584 (winver)
PowerShell Version	5.1.26100.6584 (\$PSVersionTable)
VM Hypervisor	Oracle VirtualBox v7.2.8 r173730
Lab Date	27.05.2026

2.2 User, Service Accounts & Groups

Username	Group	Role	Account Type
Ihar_Admin	SecAdmins	Security Engineer — full admin access	Human user
Analyst	Analysts	Internal analyst — scoped project access	Human user
Guest_Lab	Guests	External contractor — public read only	Human user
WebAppSvc	(standalone)	Web application — reads Projects\Public	Service account
BackupSvc	(standalone)	Nightly backup — reads entire tree	Service account
AuditLogger	(standalone)	SIEM log forwarder — writes to Logs only	Service account

Adding Human Users

Command syntax: net user <username> <password> /add

Weak or identical passwords are used here to simplify the lab. In a real-world scenario, all service and user accounts must use strong, unique credentials — and service accounts should use auto-rotating Group Managed Service Accounts (gMSA) where possible.

```
PS C:\WINDOWS\system32> net user Ihar_Admin Lab123! /add
The command completed successfully.

PS C:\WINDOWS\system32> net user Analyst Lab321! /add
The command completed successfully.

PS C:\WINDOWS\system32> net user Guest_Lab Lab111! /add
The command completed successfully.
```

Fig 1 — Human users Ihar_Admin, Analyst, and Guest_Lab created successfully

Adding Service Accounts

Service accounts are created as standard local users to replicate real-world permission concepts (same model as Group Managed Service Accounts). The same net user syntax applies.

```
PS C:\WINDOWS\system32> net user WebAppSvc Lab123! /add
The command completed successfully.

PS C:\WINDOWS\system32> net user BackupSvc Lab123! /add
The command completed successfully.

PS C:\WINDOWS\system32> net user AuditLogger Lab123! /add
The command completed successfully.
```

Fig 2 — Service accounts WebAppSvc, BackupSvc, and AuditLogger created

Creating Security Groups

Command syntax: net localgroup <group_name> /add

```
PS C:\WINDOWS\system32> net localgroup SecAdmins /add
The command completed successfully.

PS C:\WINDOWS\system32> net localgroup Analysts /add
The command completed successfully.

PS C:\WINDOWS\system32> net localgroup Guests /add
The group already exists.
```

Fig 3 — Security groups SecAdmins, Analysts, and Guests created (Guests already existed by default)

Assigning Users to Groups

Command syntax: net localgroup <group_name> <username> /add

```
PS C:\WINDOWS\system32> net localgroup SecAdmins Ihar_Admin /add
The command completed successfully.

PS C:\WINDOWS\system32> net localgroup Analysts Analyst /add
The command completed successfully.

PS C:\WINDOWS\system32> net localgroup Guests Guest /add
System error 1378 has occurred.

The specified account name is already a member of the group.

PS C:\WINDOWS\system32> net localgroup Guests Guest_Lab /add
The command completed successfully.
```

Fig 4 — Users assigned to their respective groups.

2.3 Working Directory Tree

The following folder structure was created using mkdir. PSTree module was installed for a cleaner hierarchical view. The tree below represents the full security boundary layout for NovaSec Corp.

```
Source: C:\SecureLab

Mode                Length  Hierarchy
----
d-----          0.00  B SecureLab
d-----          0.00  B   Logs
d-----          0.00  B   Projects
d-----          0.00  B       Internal
d-----          0.00  B       Public
d-----          0.00  B       Secret
d-----          0.00  B   Users
d-----          0.00  B       Analyst
d-----          0.00  B       Guest
d-----          0.00  B       Ihara_Admin
```

Fig 5 — SecureLab directory tree as displayed by Get-PSTree

2.4 Planned Permission Matrix

↓ = inherited from parent | RX = Read+Execute | M = Modify | F = Full Control | ⚡ = No access

Folder	Ihara_Admin / SecAdmins	Analysts	Guests	WebAppSvc	BackupSvc	AuditLogger	Inheritance
C:\SecureLab	F	RX	—	—	RX	—	Active — root
Projects\Public	F	RX ↓	RX	RX (expl.)	RX ↓	—	Active
Projects\Internal	F	M	—	—	RX ↓	—	Active
Projects\Secret	F	⚡ NONE	⚡ NONE	⚡ NONE	⚡ NONE	⚡ NONE	BROKEN
Users\Ihara_Admin	F	—	—	—	RX ↓	—	Explicit
Users\Analyst	F	M	—	—	RX ↓	—	Explicit
Users\Guest	F	—	RX	—	RX ↓	—	Explicit
Logs\	F	RX ↓	—	—	RX ↓	M	Active

3. Core Security Concepts

Before assigning permissions, it is important to understand the underlying model. The goal is not merely to type icacs commands correctly — it is to understand what Windows is doing internally and why each component exists.

3.1 Security Descriptors

Every securable object in Windows — files, folders, registry keys, processes, services, named pipes, access tokens — carries a data structure called a Security Descriptor (SD). It is the single source of truth for all access control on that object.

Component	Full Name	What It Contains	Lab Relevance
Owner SID	Object Owner	The SID of the user or group that owns the object. The owner can always modify the DACL regardless of other permissions.	The user who creates a folder is its owner. Visible in Get-Acl output.
Group SID	Primary Group	A primary group SID. Largely unused on modern Windows.	Typically SYSTEM or BUILTIN\Administrators. Not configured in this lab.
DACL	Discretionary ACL	The list of ACEs controlling who can and cannot access the object.	Every icacls command writes to the DACL. Covered in depth in Section 4.
SACL	System ACL	The list of audit rules — which access attempts generate Windows Event Log entries.	Not configured in this lab. Used for SIEM integration in production.

A NULL DACL (no DACL at all) means everyone has full access. An empty DACL (present but with zero ACEs) means no one has access. These behave very differently — a common source of misconfiguration in production environments.

3.2 Viewing a Security Descriptor in PowerShell

The Get-Acl cmdlet returns the full Security Descriptor of any object. Assigning the result to a variable allows multiple queries against the same object without re-executing the command.

```
PS C:\> $acl = Get-Acl "C:\SecureLab"
PS C:\> $acl | Format-List

Path      : Microsoft.PowerShell.Core\FileSystem::C:\SecureLab
Owner     : WINDOWS-11SH\ vboxuserw
Group     : WINDOWS-11SH\None
Access    : BUILTIN\Administrators Allow FullControl
           NT AUTHORITY\SYSTEM Allow FullControl
           BUILTIN\Users Allow ReadAndExecute, Synchronize
           NT AUTHORITY\Authenticated Users Allow Modify, Synchronize
           NT AUTHORITY\Authenticated Users Allow -536805376
Audit     :
Sddl      : O:S-1-5-21-1820773894-3165071869-3654933907-1000G:S-1-5-21-1820773894-3165071869-3654933907-513D:AI(A;OICIID;FA;;;BA)(A;OICIID;FA;;;SY)(A;OICIID;0x1200a9;;;BU)(A;ID;0x1301bf;;;AU)(A;OICIID;SDGXGWGR;;;AU)

PS C:\> $acl.Sddl
O:S-1-5-21-1820773894-3165071869-3654933907-1000G:S-1-5-21-1820773894-3165071869-3654933907-513D:AI(A;OICIID;FA;;;BA)(A;OICIID;FA;;;SY)(A;OICIID;0x1200a9;;;BU)(A;ID;0x1301bf;;;AU)(A;OICIID;SDGXGWGR;;;AU)
```

Fig 6 — Get-Acl output for C:\SecureLab showing Owner, Group, Access rules, and SDDL string

SDDL Breakdown

The SDDL (Security Descriptor Definition Language) string appears complex at first glance. The table below decodes each component from the output above.

Component	Meaning	Description
-----------	---------	-------------

O:	Owner	Defines the owner of the object
S-1-5-21-...-1000	Owner SID	Security Identifier of the owner account
G:	Primary Group	Defines the primary group
D:AI	DACL / Auto-Inherit	Discretionary ACL — permissions are auto-inherited from parent
(A;OICIID;FA;;;BA)	ACE Entry	Allow: Administrators (BA) have Full Access (FA) — inherited (ID)
(A;OICIID;FA;;;SY)	ACE Entry	Allow: SYSTEM (SY) has Full Access — inherited
(A;OICIID;0x1200a9;;;BU)	ACE Entry	Allow: Built-in Users (BU) have Read/Execute — inherited
(A;ID;0x1301bf;;;AU)	ACE Entry	Allow: Authenticated Users (AU) have Modify — inherited (non-propagating)
OI	Object Inherit	ACE applies to files inside the folder
CI	Container Inherit	ACE applies to subfolders inside the folder
IO	Inherit Only	ACE does not apply to the current folder itself — only to children
ID	Inherited	This ACE was inherited from a parent folder
FA	Full Access	Complete control over the object

Common SID Abbreviations

Abbreviation	Meaning
BA	Built-in Administrators
SY	SYSTEM
BU	Built-in Users
AU	Authenticated Users
WD	Everyone (World)
AN	Anonymous
DA	Domain Admins

Permission Mask Reference

Mask	Meaning
FA	Full Access
GR	Generic Read
GW	Generic Write
GX	Generic Execute

GA	Generic All
SD	Delete

3.3 Security Identifiers (SIDs) — The Real Identity Layer

Windows does not use usernames to make access decisions. Internally, every user, group, and computer is identified by a Security Identifier (SID) — a unique numeric value embedded into every ACE in every DACL.

At login, Windows builds an Access Token containing the user's SID and the SIDs of every group they belong to. When a process attempts to open a file, the kernel compares the token's SIDs against the DACL's ACEs. The username visible on screen is purely a display label — the real comparison is always SID-to-SID.

```
PS C:\> whoami /all
USER INFORMATION
-----
User Name          SID
-----
windows-11sh\vboxuserw S-1-5-21-1820773894-3165071869-3654933907-1000

GROUP INFORMATION
-----
Group Name          Type          SID          Attributes
-----
Everyone            Well-known group S-1-1-0      Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Local account and member of Administrators group Well-known group S-1-5-114    Mandatory group, Enabled by default, Enabled group
BUILTIN\Administrators Alias          S-1-5-32-544 Mandatory group, Enabled by default, Enabled group, Group
```

Fig 7 — whoami /all output showing user SID and group membership SIDs in the access token

4. Setting Permissions

Permissions are assigned using the `icacls` (Integrity Control Access Control List) command with the `/grant` option. The general syntax is:

4.1 Root Permissions

The root SecureLab folder receives Full Control for SecAdmins and Read+Execute for BackupSvc. The (OI)(CI) flags ensure both entries propagate into all files and subfolders automatically.

```
PS C:\> icacls C:\SecureLab /grant "BackupSVC:(OI)(CI)(RX)"
processed file: C:\SecureLab
Successfully processed 1 files; Failed processing 0 files
PS C:\> icacls C:\SecureLab /grant "SecAdmins:(OI)(CI)(F)"
processed file: C:\SecureLab
Successfully processed 1 files; Failed processing 0 files
```

Fig 8 — BackupSvc granted RX and SecAdmins granted Full Control on the root C:\SecureLab

4.2 Analyst and Guest Permissions

Analysts need read access across all of Projects, with Modify rights restricted to the Internal subfolder only. Guests get read access on Public exclusively.

```
PS C:\> icacls C:\Securelab\Projects /grant "Analysts:(OI)(CI)(RX)"  
processed file: C:\Securelab\Projects  
Successfully processed 1 files; Failed processing 0 files
```

Fig 9 — Analysts granted (OI)(CI)(RX) on Projects — propagates to all child objects

```
icacls C:\Securelab\Projects\Internal "Analysts:(OI)(CI)(M)"
```

Fig 10 — Analysts Modify on Internal.

```
PS C:\> echo "Our guests will only able to RX inside Public folder - no other permissions for them"; icacls C:\Securelab\Projects\Public /grant "Guests:(OI)(CI)(RX)"
```

Fig 11 — Guests granted (OI)(CI)(RX) on Projects\Public

4.3 Locking Down the Secret Folder

The Secret folder requires hard isolation — no inherited permissions from its parent chain. This is achieved by breaking the inheritance chain with /inheritance:r, then explicitly granting Full Control to Ihar_Admin only.

```
PS C:\> icacls C:\Securelab\Projects\Secret /inheritance:r  
processed file: C:\Securelab\Projects\Secret  
Successfully processed 1 files; Failed processing 0 files  
PS C:\> icacls C:\Securelab\Projects\Secret /grant "Ihar_Admin:(OI)(CI)(F)"  
processed file: C:\Securelab\Projects\Secret  
Successfully processed 1 files; Failed processing 0 files  
PS C:\> icacls C:\Securelab\Projects\Secret  
C:\Securelab\Projects\Secret WINDOWS-11SH\Ihar_Admin:(OI)(CI)(F)
```

Fig 12 — Inheritance broken on Secret; Ihar_Admin granted explicit Full Control. Verification confirms only one ACE remains.

/inheritance:r is permanent until re-enabled with /inheritance:e. After this command, even SecAdmins may lose access unless explicitly re-granted. Always verify with icacls immediately after execution.

Another handy command is /inheritance:d — it disables inheritance but copies all inherited permissions as explicit entries before removing the link. Use it when you want to preserve existing access while preventing future parent changes from propagating down.

4.4 Configuring User Home Folders

Each principal gets a scoped home folder. BackupSvc already inherits RX from the root, so only user-specific grants are needed here.

```
icacls C:\Securelab\Users\Analyst /grant "Analysts:(OI)(CI)(M)"  
icacls C:\Securelab\Users\Guest /grant "Guests:(OI)(CI)(RX)"  
  
# Ihar_Admin home: correct syntax uses /grant:r to replace, not append  
icacls C:\Securelab\Users\Ihar_Admin /grant:r "SecAdmins:(OI)(CI)(F)"
```

```
PS C:\> icacls C:\Securelab\Users\Analyst /grant "Analysts:(OI)(CI)(M)"
processed file: C:\Securelab\Users\Analyst
Successfully processed 1 files; Failed processing 0 files
PS C:\> icacls C:\Securelab\Users\Guest /grant "Guests:(OI)(CI)(RX)"
processed file: C:\Securelab\Users\Guest
Successfully processed 1 files; Failed processing 0 files
PS C:\> icacls C:\Securelab\Users\Ihar_Admin /grant "SecAdmins:(OI)(CI)(RX)"
processed file: C:\Securelab\Users\Ihar_Admin
Successfully processed 1 files; Failed processing 0 files
PS C:\> icacls C:\Securelab\Users\Ihar_Admin /grant:r "SecAdmins:(OI)(CI)(F)"
processed file: C:\Securelab\Users\Ihar_Admin
Successfully processed 1 files; Failed processing 0 files
```

Fig 13 — Home folder permissions set for Analyst (M), Guest (RX), and Ihar_Admin (F)

5. Service Account Permissions

Service accounts are one of the most common sources of real-world security vulnerabilities. An over-permissive service account can mean a compromised service escalates to full server compromise. This section applies the Principle of Least Privilege to each of the three NovaSec Corp services.

5.1 Why Service Accounts Are Different

Human User Account	Service Account
Logs in interactively — activity is visible and traceable	Runs silently in the background — harder to monitor in real time
Belongs to a named, accountable person	Shared across automation — harder to attribute to a responsible party
Logs out at end of day — attack window is limited	Runs 24/7 — the attack window is permanently open
Can be protected with MFA at login	MFA is typically not possible for automated service accounts
Password changes are a routine admin task	Password changes may break dependent services — so they rarely happen

✓ In production environments, use gMSA (Group Managed Service Accounts) or virtual service accounts (NT SERVICE\...) which receive auto-rotating passwords managed by Windows — eliminating the stale-credential risk entirely.

5.2 WebAppSvc — Internal Web Application

Requirement: The internal web application serves files from Projects\Public. It must be able to read files — nothing else. It must not list Secret, write to Internal, or access Logs.

This is an explicit grant at the lowest necessary level — the definition of least privilege.

```
PS C:\> icacls C:\Securelab\Projects\Public /grant "WebAppSvc:(OI)(CI)(RX)"
processed file: C:\Securelab\Projects\Public
Successfully processed 1 files; Failed processing 0 files
```

Fig 14 — WebAppSvc granted Read+Execute on Projects\Public only

5.3 BackupSvc — Nightly Backup Service

Requirement: The backup job must read every file on the server for off-site transfer. It must never write, never modify permissions, and never take ownership. Read+Execute — propagated through the entire tree from the root.

BackupSvc was already granted at the root in Section 4.1. The screenshot below recaps that syntax for reference.

```
PS C:\> icacls C:\SecureLab /grant "BackupSVC:(OI)(CI)(RX)"
processed file: C:\SecureLab
Successfully processed 1 files; Failed processing 0 files
```

Fig 15 — BackupSvc RX grant at root — propagates via (OI)(CI) to all children except Secret

BackupSvc cannot read Secret — this is a deliberate design decision, not an oversight. Sensitive data in a properly isolated folder should not be accessible to the backup service account. Real backups of isolated data use dedicated mechanisms such as VSS shadow copies or a privileged Backup Operator group account.

5.4 AuditLogger — SIEM Log Forwarder

Requirement: The audit logging service writes collected events to C:\SecureLab\Logs. It needs Modify on Logs only — no access anywhere else in the tree.

```
PS C:\> icacls C:\Securelab\Logs /grant "AuditLogger:(OI)(CI)(M)"
processed file: C:\Securelab\Logs
Successfully processed 1 files; Failed processing 0 files
```

Fig 16 — AuditLogger granted Modify on Logs folder

```
PS C:\> icacls C:\Securelab\Projects /deny "AuditLogger:(OI)(CI)(RX)"
processed file: C:\Securelab\Projects
Successfully processed 1 files; Failed processing 0 files
```

Fig 17 — Explicit DENY ACE applied: AuditLogger cannot access Projects under any circumstances

Explicit Deny ACEs are powerful but should be used deliberately. An explicit Deny on a parent can block access you intended to grant on a child. Document every Deny ACE thoroughly — they are among the hardest permission issues to debug in production.

5.5 Service Account Permission Summary

Service Account	Granted Path	Rights	Rationale
WebAppSvc	Projects\Public	RX (explicit)	Serves public files only — no deeper access required
BackupSvc	C:\SecureLab (root)	RX (explicit, propagates)	Reads entire tree via inheritance — Secret blocked by inheritance break

AuditLogger	Logs\	Modify (explicit)	Writes log files to Logs folder only
AuditLogger	Projects\	DENY RX (explicit)	Hard deny — cannot escape Logs even if added to another group

6. Validation

Validation is not optional — it is the proof that the lab is correctly configured. Security engineering means verifying outcomes, not assuming them.

6.1 Root Folder Validation

Running `icacls` directly on the root gives a quick sanity check. The output should show BackupSvc (OI)(CI)(RX) and SecAdmins (OI)(CI)(F) as explicit entries, with all system defaults as inherited (I) entries.

```
PS C:\> icacls C:\Securelab
C:\Securelab BUILTIN\BackupSvc:(OI)(CI)(RX)
C:\Securelab BUILTIN\SecAdmins:(OI)(CI)(F)
C:\Securelab BUILTIN\Administrators:(I)(OI)(CI)(F)
C:\Securelab NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
C:\Securelab BUILTIN\Users:(I)(OI)(CI)(RX)
C:\Securelab NT AUTHORITY\Authenticated Users:(I)(M)
C:\Securelab NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

Fig 18 — `icacls C:\SecureLab` confirming BackupSvc RX and SecAdmins Full Control as explicit entries

6.2 Full Recursive ACL Snapshot

The following command performs a recursive walk of the entire SecureLab tree and exports all ACL data to a text file for review and documentation.

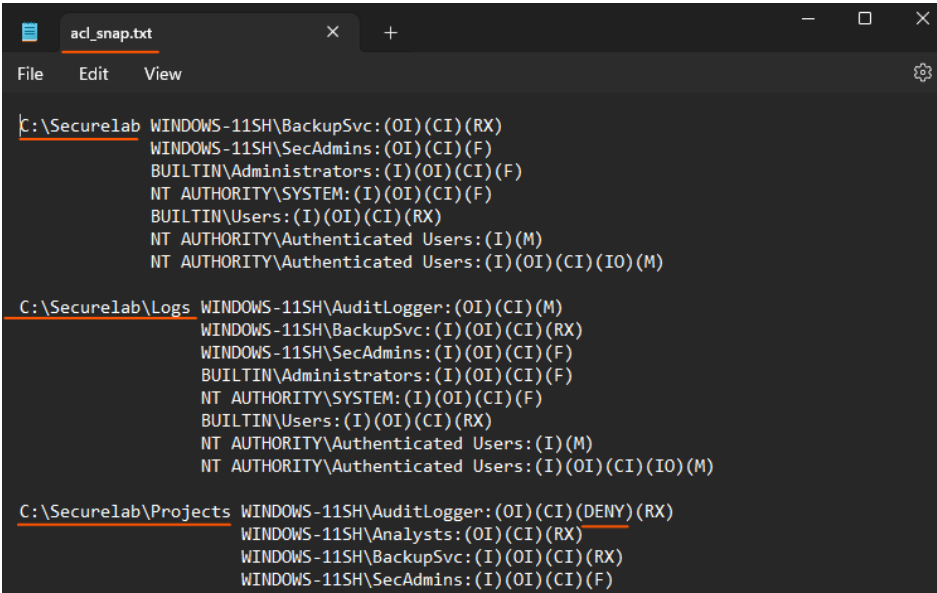
Component	Meaning
<code>icacls C:\SecureLab</code>	Display ACLs for the root folder
<code>/T</code>	Recurse into all descendants — files and subfolders
<code>></code>	Redirect standard output
<code>acl_snap.txt</code>	Save the full ACL report to this file

```
PS C:\> icacls C:\Securelab /T > C:\Securelab\Logs\acl_snap.txt
C:\Securelab\Projects\Secret\*: Access is denied.
PS C:\>
```

Fig 19 — Full recursive ACL export to `acl_snap.txt`. Note 'Access is denied' on `Projects\Secret` — confirming the inheritance break is working correctly.

```
PS C:\> notepad C:\Securelab\Logs\acl_snap.txt
```

Fig 20 — Opening `acl_snap.txt` in Notepad



```

acl_snap.txt
File Edit View

C:\Securelab WINDOWS-11SH\BackupSvc:(OI)(CI)(RX)
               WINDOWS-11SH\SecAdmins:(OI)(CI)(F)
               BUILTIN\Administrators:(I)(OI)(CI)(F)
               NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
               BUILTIN\Users:(I)(OI)(CI)(RX)
               NT AUTHORITY\Authenticated Users:(I)(M)
               NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)

C:\Securelab\Logs WINDOWS-11SH\AuditLogger:(OI)(CI)(M)
                  WINDOWS-11SH\BackupSvc:(I)(OI)(CI)(RX)
                  WINDOWS-11SH\SecAdmins:(I)(OI)(CI)(F)
                  BUILTIN\Administrators:(I)(OI)(CI)(F)
                  NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                  BUILTIN\Users:(I)(OI)(CI)(RX)
                  NT AUTHORITY\Authenticated Users:(I)(M)
                  NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)

C:\Securelab\Projects WINDOWS-11SH\AuditLogger:(OI)(CI)(DENY)(RX)
                     WINDOWS-11SH\Analysts:(OI)(CI)(RX)
                     WINDOWS-11SH\BackupSvc:(I)(OI)(CI)(RX)
                     WINDOWS-11SH\SecAdmins:(I)(OI)(CI)(F)

```

Fig 21 — `acl_snap.txt` contents in Notepad. Note: the DENY rule for AuditLogger appears first on the Projects line — Windows always processes explicit Deny ACEs before Allow ACEs regardless of their position in the list.

✓ The 'Access is denied' line for Projects\Secret is a positive result — it confirms that the inheritance break is working and that the recursive walk cannot enumerate the contents of the isolated folder.

7. Security Observations

Observation 1 — Inheritance Break Is a Double-Edged Sword

The `/inheritance:r` pattern used on `Secret` creates a clean, explicit-only DACL — the most secure configuration for sensitive data. The same command, however, can silently remove access for service accounts and admin groups that were only covered by their inherited entry.

- Risk: After `/inheritance:r`, the administrator forgets to re-grant the backup service or a key admin group. Backup silently stops functioning. The gap is only discovered after a failed restore attempt in an incident.
- Mitigation: Always run `icacls` on the folder immediately after `/inheritance:r`. Document every explicit grant added directly afterward.

`/inheritance:d` is worth knowing as an alternative — it disables the inheritance link but converts all existing inherited entries to explicit ones before removing the connection. This prevents future parent ACL changes from propagating down while preserving current effective access.

Observation 2 — Service Accounts Are Persistent Attack Surfaces

Human users log out. Service accounts run continuously, with static credentials and no multi-factor authentication. In this lab, `WebAppSvc` has read access to `Projects\Public`. If the web application contains a path traversal vulnerability, an attacker could leverage `WebAppSvc`'s access context to read any file the account can reach.

- Risk: A misconfigured or compromised service account grants an attacker persistent, low-noise access — harder to detect than an interactive login.
- Mitigation: Scope service accounts to the minimum required path and permission. Use `NT SERVICE\` virtual accounts where possible. Monitor for unusual access patterns via SACL audit logging and SIEM correlation rules.

Observation 3 — Group-Based ACLs Scale; User-Based ACLs Are a Liability

This lab uses groups (`SecAdmins`, `Analysts`, `Guests`) for role-based access control, mirroring real enterprise Active Directory design. The only user-direct ACE in the lab is `Ihar_Admin:(F)` on `Secret` — acceptable for an admin-only isolated folder.

- Scalable: Adding a new analyst requires only adding them to the `Analysts` group. Zero folder permission changes needed.
- Auditable: `Analysts:(M)` in an ACL immediately conveys intent to any reviewer.
- Anti-pattern: Granting `Ihar_Admin:(F)` directly on every folder means adding a second admin account requires modifying every ACL individually — an operational and audit nightmare at scale.

8. Conclusion

This lab built a complete enterprise-style access control system from scratch. More importantly, it provides understanding of the architecture underneath the commands — Security Descriptors, SID-based identity, service account threat modelling, and the attack surfaces that emerge from misconfigurations.

What This Lab Demonstrated

- Designed a multi-tier ACL hierarchy for a file server with distinct security zones
- Applied the Principle of Least Privilege to both human users and automated service accounts
- Used `/inheritance:r` to create hard ACL isolation on sensitive data
- Explained the full Security Descriptor model: Owner, Group, DACL, SACL
- Implemented and justified an explicit DENY ACE as a defence-in-depth control
- Documented all findings in a professional security engineering format with command evidence

The End.
